TALLER 1

Uso de la Inteligencia Artificial

Low Code Engineering

Maestría en Inteligencia Artificial · USFQ
Abril 2025

Integrantes – Grupo 5

Integrante	Problema asignado
Nancy Altamirano	A. TSP – Travelling Salesman Problem
Gustavo Berru	A. TSP – Travelling Salesman Problem
Raquel Pacheco	B. El acertijo del granjero y el bote
Kevin Viteri	C. La Torre de Hanoi

Contenido

- 1. Problema A — TSP (Travelling Salesman Problem)
- 2. Problema B — El acertijo del granjero y el bote
- 3. Problema C — La Torre de Hanoi
- 4. Conclusiones generales

A. TSP — Travelling Salesman Problem

Descripción del problema

Dada una lista de ciudades y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y regresa al origen? Se trabajó con 10 ciudades del Ecuador: Quito, Guayaquil, Cuenca, Manta, Ambato, Loja, Esmeraldas, Riobamba, Ibarra y Latacunga.

Representación del problema

El TSP se representa como un **grafo completo ponderado** G = (V, E) donde V son las ciudades y cada arista (i,j) ∈ E tiene peso igual a la distancia en km. El objetivo es encontrar el ciclo hamiltoniano de menor peso total.

Algoritmos implementados

Algoritmo	Descripción	Complejidad
Nearest Neighbor	Greedy: parte de Quito y siempre salta al vecino más cercano no visitado.	$O(n^2)$
ACO	Colonia de hormigas: 25 hormigas × 150 iteraciones con feromonas, evaporación ρ=0.15 y élite.	$O(n^2)$
2-opt	Post-proceso: invierte segmentos de la ruta ACO hasta que no haya mejora posible.	$O(n^2)$

Fórmula de transición ACO

Cada hormiga elige el siguiente nodo j desde el nodo i con probabilidad:

```
p(i→j) = [ τ(i,j)^α · η(i,j)^β ] / Σk [ τ(i,k)^α · η(i,k)^β ]
```

donde τ son las feromonas (memoria colectiva), η = 1/dist (heurística de distancia), α=1.2 pondera la explotación y β=2.5 la exploración. Tras cada iteración se evapora: τ ← (1−ρ)·τ y las 3 hormigas élite refuerzan la mejor ruta global.

Visualizaciones — TSP

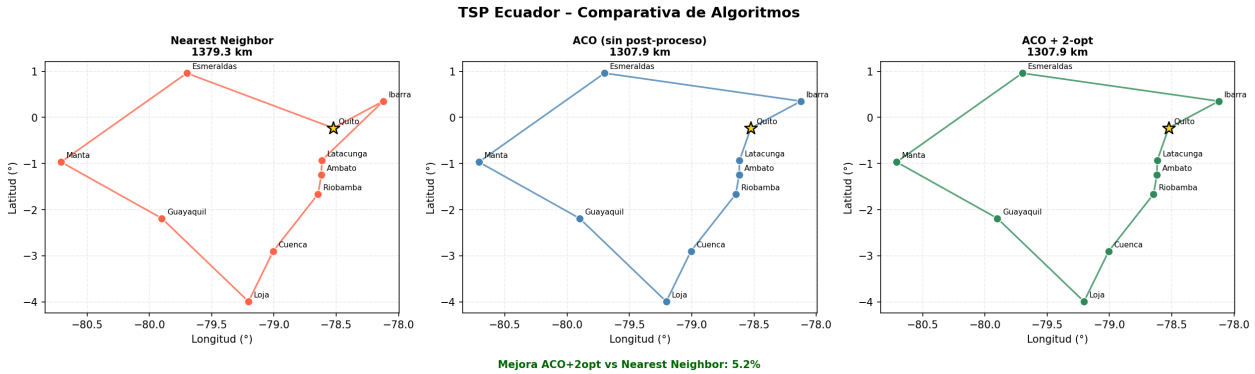
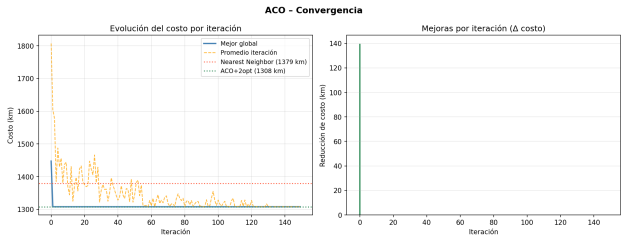


Fig. 1 — Comparativa de rutas: Nearest Neighbor (rojo, ★=inicio) vs ACO (azul) vs ACO+2opt (verde). La mejora sobre el baseline greedy es del 5.2%.



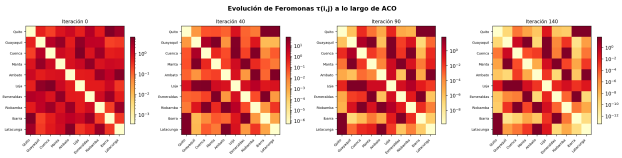
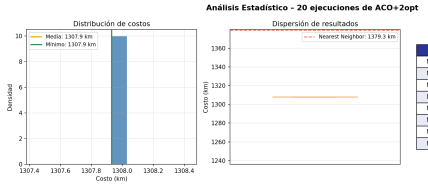


Fig. 2 (izq.) — Convergencia: mejor global y promedio por iteración. Fig. 3 (der.) — Feromonas τ(i,j) en escala logarítmica en 4 momentos del algoritmo (el algoritmo aprende qué arcos son buenos).



Estadísticas descriptivas

Métrica	Valor
Mínimo	1307.93 km
Máximo	1307.93 km
Media	1307.93 km
Desv. estándar	0.00 km
Mediana	1307.93 km
Nearest Neighbor	1379.26 km
Mejora media vs NN	5.2%
Mejora máx vs NN	5.2%

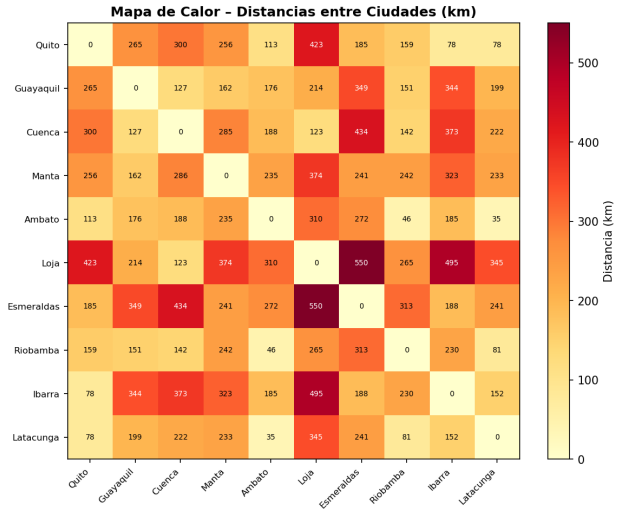


Fig. 4 (izq.) — Análisis estadístico de 20 ejecuciones independientes: media=1307.93 km, desv.est.=0 (el algoritmo converge siempre al mismo óptimo). Fig. 5 (der.) — Mapa de calor de distancias entre ciudades.

Conclusión — TSP

El TSP es el problema más complejo del taller: es NP-difícil, sin solución exacta eficiente para n grande. ACO lo resuelve como metaheurística bioinspirada, combinando memoria colectiva (feromonas) con exploración local (2-opt). La desviación estándar de 0 en 20 corridas demuestra que la instancia de 10 ciudades tiene un óptimo muy bien definido y ACO lo encuentra de forma consistente.

B. El acertijo del granjero y el bote

Descripción del problema

Un granjero debe cruzar un río con un lobo, una cabra y una col. La barca solo admite al granjero y un ítem. Restricciones: lobo y cabra no pueden quedarse solos (el lobo come la cabra), ni cabra y col (la cabra come la col).

Representación como espacio de estados

Cada situación se codifica como una tupla de 4 bits:

```
estado = (granjero, lobo, cabra, col) donde 0 = orilla izquierda, 1 = derecha
```

Estado inicial: (0,0,0,0) → Estado objetivo: (1,1,1,1) El espacio total tiene 2■=16 estados posibles; solo 10 son válidos tras descartar los que violan las restricciones.

Algoritmo: BFS (Búsqueda en Anchura)

BFS explora el grafo de estados nivel por nivel, garantizando encontrar la solución con el menor número de pasos. Desde cada estado, el granjero puede cruzar solo o llevando uno de los tres ítems que estén en su mismo lado; solo se encolan los estados válidos.

Visualizaciones — Granjero

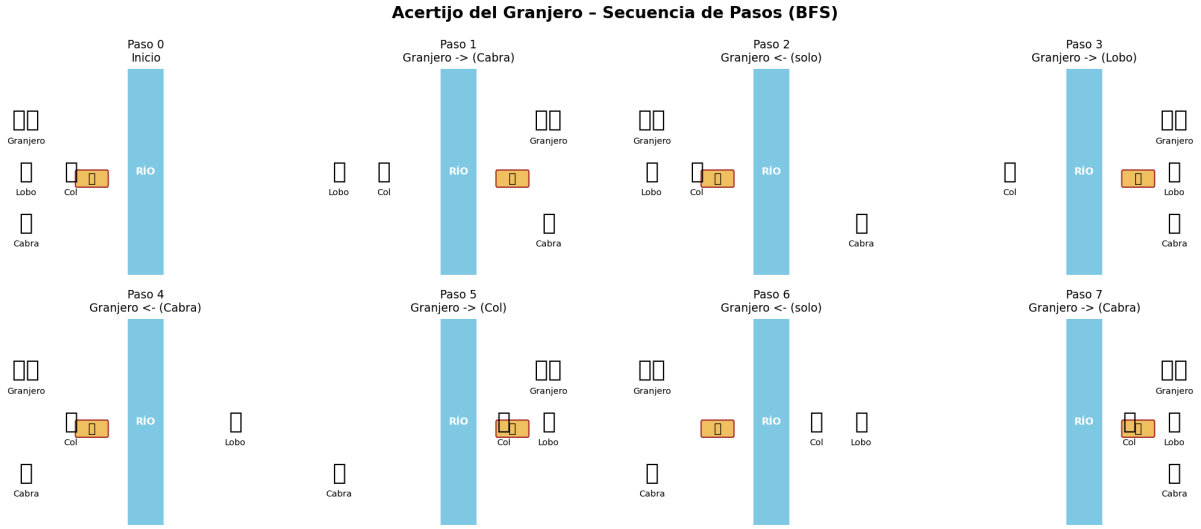


Fig. 6 — Secuencia completa de 7 pasos de la solución óptima. Cada panel muestra las dos orillas, la posición de cada elemento y la barca. Solución: llevar cabra | regresar solo | llevar lobo | regresar con cabra | llevar col | regresar solo | llevar cabra.

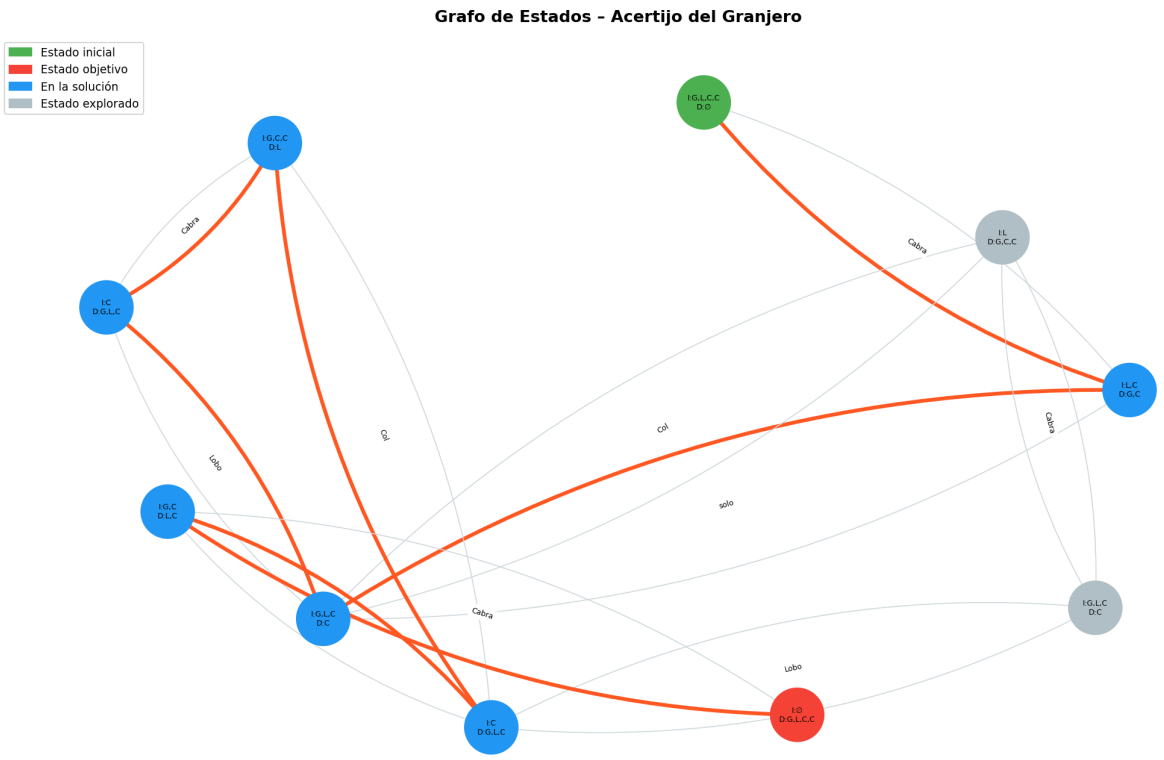


Fig. 7 — Grafo completo de transiciones de estados. Verde = estado inicial, rojo = estado objetivo, azul = nodos en la solución, gris = estados explorados pero no usados. La ruta óptima se resalta en naranja.

Conclusión — Granjero

El acertijo del granjero es el problema más elegante del taller para ilustrar búsqueda en espacios de estados. Con solo 10 estados válidos, BFS garantiza la solución óptima en exactamente 7 movimientos. La clave del modelado es definir correctamente la función de validez del estado, que descarta situaciones peligrosas antes de encolarlas.

C. La Torre de Hanoi

Descripción del problema

La Torre de Hanoi es un clásico de la informática teórica. Se tienen **n discos** de distintos tamaños apilados en la Torre A (el más grande abajo), y dos torres vacías B y C. El objetivo es trasladar *todos* los discos a la Torre C siguiendo dos reglas: (1) solo se mueve un disco a la vez, y (2) nunca se coloca un disco más grande sobre uno más pequeño. El problema fue formulado por Édouard Lucas en 1883 y sigue siendo referencia obligatoria en cursos de algoritmos porque exhibe una estructura recursiva perfectamente simétrica.

Estrategia: Divide y Vencerás

La clave es reconocer que mover n discos de A a C es equivalente a tres subproblemas independientes. Esta descomposición es posible porque el disco más grande es completamente independiente de los n–1 discos que están encima de él:

- **Paso 1:** Mover los n–1 discos superiores de A hacia B, usando C como auxiliar.
- **Paso 2:** Mover el disco más grande (disco n) directamente de A hacia C (un solo movimiento).
- **Paso 3:** Mover los n–1 discos de B hacia C, usando A como auxiliar.

Los pasos 1 y 3 son instancias del *mismo problema* con n–1 discos, lo que permite aplicar la misma estrategia recursivamente. El caso base es trivial: con n=1 disco, se mueve directamente de origen a destino sin ninguna complejidad.

Algoritmo recursivo

El pseudocódigo captura la estrategia de manera exacta. La función recibe el número de discos y los nombres de las tres torres (origen, destino, auxiliar):

```
def hanoi(n, origen, destino, auxiliar): # Caso base: mover directamente if n == 1: mover
disco de origen -> destino return # Paso 1: liberar el disco grande hanoi(n-1, origen,
auxiliar, destino) # Paso 2: mover el disco mas grande mover disco de origen -> destino #
Paso 3: reconstruir sobre el disco grande hanoi(n-1, auxiliar, destino, origen)
```

Obsérvese que los roles de las torres **rotan** en cada llamada recursiva: lo que era destino pasa a ser auxiliar y viceversa. Este intercambio es lo que garantiza que nunca se violen las restricciones del problema.

Árbol de recursión (n = 3)

Con n=3, la ejecución genera 7 movimientos. El árbol de llamadas ilustra cómo se descompone el problema:

```
hanoi(3, A, C, B) +--> hanoi(2, A, B, C) | +--> hanoi(1, A, C, B) => mueve disco 1: A -> C
| +--> mover disco 2: A -> B | +--> hanoi(1, C, B, A) => mueve disco 1: C -> B +--> mover
disco 3: A -> C +--> hanoi(2, B, C, A) +--> hanoi(1, B, A, C) => mueve disco 1: B -> A +-->
mover disco 2: B -> C +--> hanoi(1, A, C, B) => mueve disco 1: A -> C
```

Cada nivel del árbol duplica el trabajo: el nivel 0 tiene 1 llamada, el nivel 1 tiene 2, el nivel 2 tiene 4, hasta llegar a las 2■■¹ llamadas base que corresponden a los movimientos reales.

Demostración matemática de la complejidad

Sea T(n) el número de movimientos necesarios para n discos. La recurrencia se formula directamente desde el algoritmo:

```
T(1) = 1 (caso base) T(n) = 2 * T(n-1) + 1 (2 subproblemas de n-1 + 1 movimiento del disco grande)
```

Desarrollando la recurrencia por sustitución repetida:

```
T(n) = 2*T(n-1) + 1 = 2*(2*T(n-2) + 1) + 1 = 4*T(n-2) + 3 = 4*(2*T(n-3) + 1) + 3 = 8*T(n-3) + 7 = ... = 2^(n-1) * T(1) + (2^(n-1) - 1) = 2^(n-1) + 2^(n-1) - 1 = 2^n - 1
```

Este resultado es también un **mínimo absoluto**: se puede demostrar por inducción que cualquier secuencia válida de movimientos requiere al menos 2■–1 pasos, lo que convierte al algoritmo recursivo en óptimo.

Tabla de complejidad: discos vs movimientos

Discos (n)	Movimientos (2■–1)	Tiempo estimado (1 mov/s)
1	1	1 s
2	3	3 s
3	7	7 s
4	15	15 s
5	31	31 s
6	63	1 min 3 s
7	127	2 min 7 s
8	255	4 min 15 s
9	511	8 min 31 s
10	1,023	~17 min
11	2,047	~34 min
12	4,095	~1.1 h
13	8,191	~2.3 h
14	16,383	~4.6 h
15	32,767	~9.1 h
16	65,535	~18.2 h
17	131,071	~36.4 h
18	262,143	~3.0 días
19	524,287	~6.0 días
20	1,048,575	~12.0 días

Diseño orientado a objetos (OOP)

La solución se implementa con cinco clases que siguen el principio de *responsabilidad única*:

Clase	Responsabilidad	Métodos clave
Disk	Entidad: disco con tamaño y color	color (property)
Tower	Pila de discos; valida restricciones	push(disk), pop(), top

HanoiState	Instantánea de las tres torres	snapshot()
HanoiSolver	Ejecuta la recursión y registra movimientos	_recurse(n, towers, src, dst, aux)
HanoiVisualizer	Genera animación y gráficos de análisis	animate(), plot_analysis(), save_all()

La clase **Tower.push()** lanza ValueError si se intenta colocar un disco mayor sobre uno menor, lo que convierte la restricción del puzzle en una invariante verificada en tiempo de ejecución. **HanoiSolver._recurse()** opera directamente sobre las instancias Tower, mientras que **HanoiState** captura una copia profunda de las tres torres en cada paso para permitir la visualización completa.

Visualizaciones — Hanoi

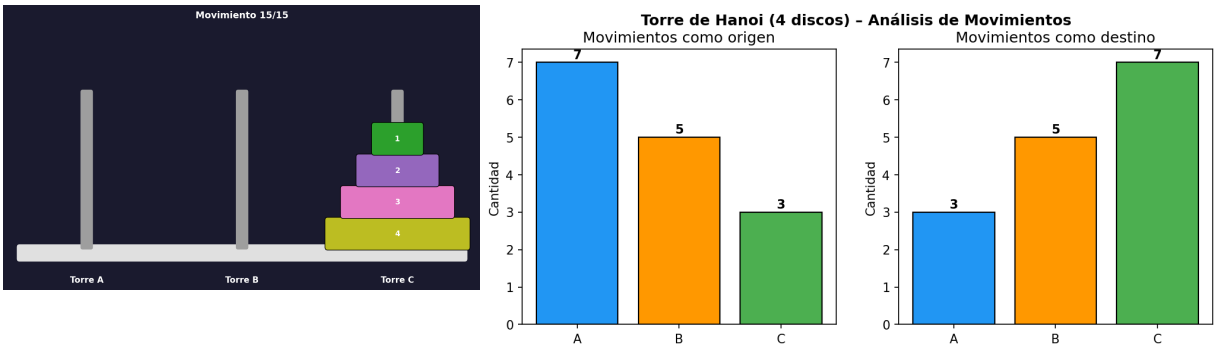


Fig. 8 (izq.) — Estado final con 4 discos: todos apilados en Torre C en orden correcto. Fig. 9 (der.) — Distribución de movimientos: Torre A (origen) genera 8 salidas, Torre C (destino) recibe 8 entradas, Torre B (auxiliar) equilibra el tráfico.

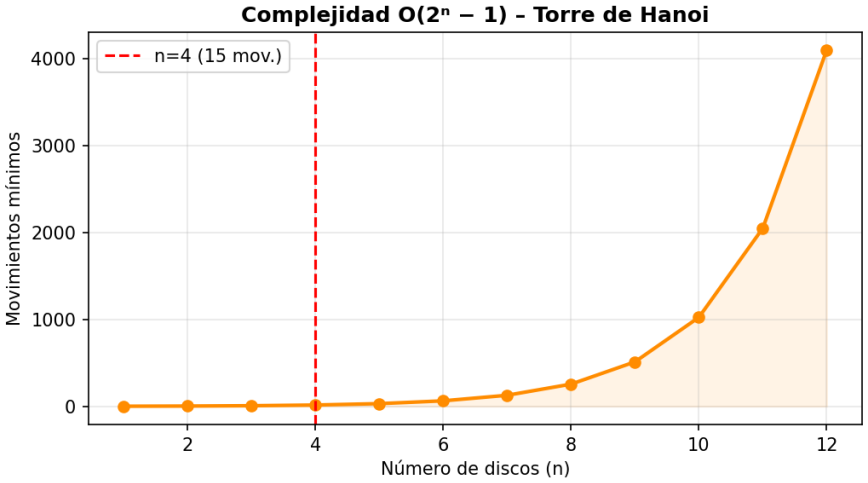


Fig. 10 — Curva de complejidad O(2■ – 1). Con 4 discos se necesitan 15 movimientos; con 10 discos serían 1023; con 20 discos más de 1 millón.

Conclusión — Hanoi

La Torre de Hanoi demuestra tres principios fundamentales de la algoritmia: (1) **Recursión óptima** — el algoritmo encuentra la solución mínima sin necesidad de exploración adicional; (2) **Complejidad exponencial inevitable** — T(n) = 2■–1 no es una debilidad del algoritmo sino un límite inferior del problema; y (3) **Separación de responsabilidades en OOP** — al distribuir la lógica entre Disk, Tower, HanoiSolver y HanoiVisualizer, el código es legible, testeable y extensible (por ejemplo, para 10 discos o para nuevas reglas de movimiento) sin modificar ninguna clase existente.

4. Conclusiones Generales

• **Representación es clave:** La dificultad de cada problema no radica en la implementación sino en elegir la representación correcta. El TSP como grafo ponderado, el Granjero como espacio de estados y Hanoi como árbol de recursión permiten aplicar algoritmos estándar directamente.

• **La IA aprovecha estructura:** ACO no resuelve el TSP por fuerza bruta sino explotando la estructura del problema: las feromonas acumulan experiencia colectiva y la heurística de distancia guía la exploración. Esto es central en IA: usar conocimiento del dominio para reducir el espacio de búsqueda.

• **Visualización como diagnóstico:** Los mapas de feromonas revelan qué aprendió el algoritmo; la curva de convergencia muestra cuándo dejó de mejorar; el grafo de estados del Granjero hace visible por qué algunas rutas son inválidas. Visualizar no es decoración, es comprensión.

• **Complejidad vs. exactitud:** TSP es NP-difícil (no hay solución exacta eficiente), Granjero tiene 16 estados (BFS exacto en ms) y Hanoi tiene solución matemática cerrada. La IA escoge el algoritmo según la naturaleza del problema, no un único enfoque universal.

• **Herramientas Low-Code:** La IA generativa (Claude, ChatGPT, Copilot) permite desarrollar soluciones de calidad académica con ciclos rápidos de prototipado, pero el analista debe validar los resultados, comprender los algoritmos y ajustar los parámetros según el contexto del problema.

Ver también: **ensayo_chips_analogicos.pdf** — La Evolución de la IA con Chips Analógicos (Grupo 5, Abril 2025)